

AI BASED HIRING PREDICTION SYSTEM

Project Report

Submitted by: Vaishnavi Sharma

3Skill AI/ML Internship Program

June 2026- August 2026

DECLARATION

I hereby declare that the project report entitled "**AI-Based Hiring Prediction System**" is an original work carried out by me as part of my internship under **3Skill**. This project has been completed using the dataset and guidelines provided during the internship program.

I further declare that the work presented in this report has been carried out by me independently with guidance wherever necessary. The project has not been submitted, either in whole or in part, to any other institution or organization for the award of any degree, diploma, or certification.

I have made every effort to ensure that the information, analysis, and results presented in this report are accurate and based on the work performed during the project. All external resources, software libraries, and references used in this project have been properly acknowledged.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **3Skill** for providing me with the opportunity to undertake this internship and work on the project titled "**AI-Based Hiring Prediction System.**" This project has provided me with valuable practical experience in applying Machine Learning techniques to solve a real-world recruitment problem.

I am thankful to the mentors and trainers at **3Skill** for their continuous guidance, support, and valuable learning resources throughout the internship. Their encouragement and constructive feedback helped me understand the complete machine learning workflow, from data preprocessing and exploratory data analysis to model development, evaluation, and optimization.

I would also like to thank everyone who contributed directly or indirectly to the successful completion of this project. Their support and motivation played an important role in enhancing my technical knowledge and problem-solving skills.

Finally, I express my gratitude to my family and well-wishers for their constant encouragement, patience, and support throughout the internship. Their motivation inspired me to complete this project with dedication and confidence.

I sincerely acknowledge all those who have contributed to the successful completion of this project.

ABSTRACT

Recruitment is a crucial process for every organization, but manually screening a large number of resumes is time-consuming and may lead to inconsistent hiring decisions. Machine Learning provides an effective solution by automating candidate evaluation based on resume-related information, thereby improving the efficiency and reliability of the recruitment process.

This project presents an **AI-Based Hiring Prediction System** developed using a synthetic recruitment dataset provided by **3Skill** during the internship program. The dataset contains information such as candidate skills, work experience, education, certifications, job role, salary expectations, and project count. The objective of the project is to predict whether a candidate is likely to be **Hired** or **Rejected** using supervised machine learning techniques.

The project follows a complete end-to-end Machine Learning workflow, including exploratory data analysis, data preprocessing, feature engineering, model training, evaluation, and optimization. Three classification algorithms—**Logistic Regression**, **K-Nearest Neighbors (KNN)**, and **Decision Tree**—were implemented and compared using evaluation metrics such as Accuracy, Precision, Recall, F1-Score, Confusion Matrix, and Classification Report. The best-performing model was further optimized using **GridSearchCV** to improve its predictive performance. In addition, feature importance analysis was carried out to identify the resume attributes that contribute most to hiring decisions.

The developed system demonstrates the practical application of Machine Learning in recruitment by providing an automated approach to candidate screening. It highlights the importance of proper data preprocessing, model evaluation, and feature selection in building reliable AI-based decision-support systems for Human Resource (HR) analytics.

TABLE OF CONTENT

CHAPTER	TITILE	PAGE NO.
1.	Introduction	1
2.	Problem Statement	2
3.	Objectives	3
4.	Dataset Description	4
5.	Methodology	5
6.	Exploratory Data Analysis (EDA)	6
7.	Data Preprocessing	10
8.	Model Building	12
9.	Model Comparison and Hyperparameter tuning	15
10.	Feature Analysis	17
11.	Conclusion	18
12.	Future scope	19
13.	References	20

INTRODUCTION

Recruitment is one of the most important processes in any organization, as selecting the right candidate directly influences productivity and business growth. With the increasing number of job applications received for every vacancy, manually screening resumes has become a time-consuming and resource-intensive task. Human recruiters often need to evaluate hundreds of resumes based on factors such as skills, work experience, educational qualifications, certifications, projects, and salary expectations before making a hiring decision.

Artificial Intelligence (AI) and Machine Learning (ML) have transformed the recruitment process by enabling automated resume screening and candidate evaluation. These technologies help organizations reduce manual effort, improve the consistency of hiring decisions, and identify suitable candidates more efficiently. By analyzing historical hiring patterns, machine learning models can learn the relationship between candidate attributes and recruitment outcomes, making them valuable decision-support tools for Human Resource (HR) professionals.

The AI-Based Hiring Prediction System developed in this project aims to predict whether a candidate is likely to be Hired or Rejected based on resume-related information. The project uses a synthetic resume dataset provided by 3Skill during the internship program. Various machine learning algorithms, including Logistic Regression, K-Nearest Neighbors (KNN), and Decision Tree, are trained and evaluated to identify the most suitable model for hiring prediction. Hyperparameter tuning and feature importance analysis are also performed to improve model performance and understand the factors that influence recruitment decisions.

This project demonstrates the complete end-to-end machine learning workflow, including data preprocessing, exploratory data analysis, feature engineering, model training, evaluation, optimization, and interpretation of results. The developed system provides valuable insights into AI-driven recruitment and highlights the potential of machine learning in modern HR analytics.

PROBLEM STATEMENT

In modern recruitment, organizations receive a large number of job applications for every available position. Manually reviewing each resume is time-consuming, labor-intensive, and may lead to inconsistent hiring decisions due to human bias or oversight. Recruiters must evaluate candidates based on multiple factors such as technical skills, work experience, educational qualifications, certifications, completed projects, and salary expectations, making the selection process increasingly complex.

To address these challenges, there is a need for an intelligent system that can automatically analyze candidate information and assist recruiters in identifying suitable applicants. Machine Learning provides an effective solution by learning patterns from historical recruitment data and predicting hiring outcomes based on candidate attributes.

The objective of this project is to develop an AI-Based Hiring Prediction System that predicts whether a candidate is likely to be Hired or Rejected using resume-related information. The system applies data preprocessing, exploratory data analysis, feature engineering, and multiple machine learning algorithms to build an accurate prediction model. The performance of different models is evaluated and compared using standard classification metrics, and the best-performing model is further optimized through hyperparameter tuning.

This project demonstrates how Artificial Intelligence can support Human Resource (HR) professionals by improving the efficiency, consistency, and reliability of the recruitment process.

OBJECTIVES

The primary objective of this project is to develop an AI-Based Hiring Prediction System that can predict whether a candidate is likely to be Hired or Rejected based on resume-related information using Machine Learning techniques.

The specific objectives of this project are:

- To understand and analyze the hiring prediction dataset provided by 3Skill.
- To perform Exploratory Data Analysis (EDA) for understanding the dataset, identifying missing values, and examining the distribution of features.
- To preprocess the data by handling missing values, removing unnecessary columns, and encoding categorical variables.
- To split the dataset into training and testing sets for model development and evaluation.
- To build multiple Machine Learning classification models, including Logistic Regression, K-Nearest Neighbors (KNN), and Decision Tree.
- To evaluate the performance of each model using Accuracy, Precision, Recall, F1-Score, Confusion Matrix, and Classification Report.
- To compare the performance of different machine learning models and identify the best-performing model.
- To optimize the selected model using GridSearchCV for improved prediction performance.
- To analyze feature importance and identify the factors that have the greatest influence on hiring decisions.
- To demonstrate a complete end-to-end Machine Learning workflow for an AI-based recruitment system.

DATASET DESCRIPTION

The dataset used in this project was provided by 3Skill as part of the internship program. It is a synthetic recruitment dataset designed to simulate a real-world resume screening process. The dataset contains information about candidates, including their technical skills, work experience, educational qualifications, certifications, applied job role, salary expectations, project count, and recruiter decisions.

The objective of using this dataset is to train a machine learning model that can predict whether a candidate is likely to be Hired or Rejected based on the information available in the resume.

Dataset Overview :

ATTRIBUTE	DESCRIPTION
Dataset Source	Internship dataset provided by 3Skill
Total Records	1000
Total Features	11 Columns
Target Variable	Recruiter Decision (Hire / Reject)
Missing Values	Present only in the Certifications column
Dataset Type	Synthetic Resume Screening Dataset

Dataset Features:

Feature Name	Description
Resume_ID	Unique identifier for each candidate
Name	Candidate's name
Skills	Technical skills mentioned in the resume

Experience (Years)	Total years of work experience
Education	Highest educational qualification
Certifications	Professional certifications obtained by the candidate
Job Role	Position applied for
Recruiter Decision	Hiring decision (Target Variable)
Salary Expectation (\$)	Expected salary of the candidate
Projects Count	Number of completed projects
AI Score (0–100)	AI-generated evaluation score

Target Variable :

The target variable for this project is Recruiter Decision, which indicates whether a candidate is selected or rejected during the hiring process.

For machine learning model development, the target variable was encoded as:

- Hire → 0
- Reject → 1

...

	Resume_ID	Name	Skills	Experience (Years)	Education	Certifications	Job Role	Recruiter Decision	Salary Expectation (\$)	Projects Count	AI Score (0-100)
0	1	Ashley Ali	TensorFlow, NLP, Pytorch	10	B.Sc	NaN	AI Researcher	Hire	104895	8	100
1	2	Wesley Roman	Deep Learning, Machine Learning, Python, SQL	10	MBA	Google ML	Data Scientist	Hire	113002	1	100
2	3	Corey Sanchez	Ethical Hacking, Cybersecurity, Linux	1	MBA	Deep Learning Specialization	Cybersecurity Analyst	Hire	71766	7	70
3	4	Elizabeth Carney	Python, Pytorch, TensorFlow	7	B.Tech	AWS Certified	AI Researcher	Hire	46848	0	95
4	5	Julie Hill	SQL, React, Java	4	PhD	NaN	Software Engineer	Hire	87441	9	100

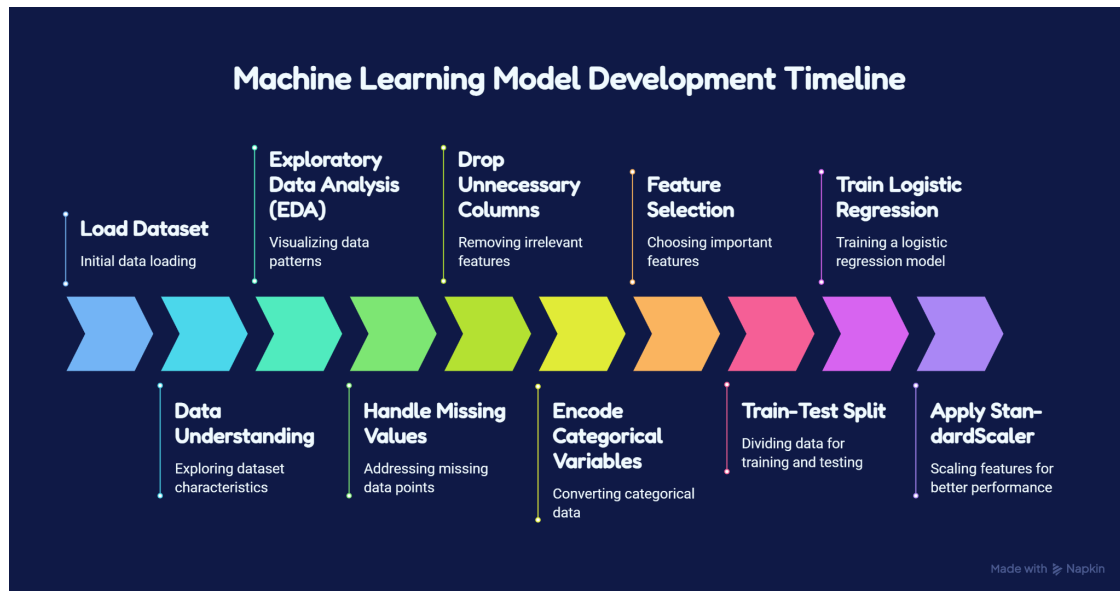
METHODOLOGY

The AI-Based Hiring Prediction System was developed using a structured Machine Learning workflow. The project began with loading and understanding the dataset provided by **3Skill**, followed by data preprocessing to prepare the data for model training. Missing values were handled, unnecessary columns were removed, and categorical variables were encoded into numerical values suitable for machine learning algorithms.

The processed dataset was then divided into training and testing sets using a stratified train-test split to maintain the class distribution. Three classification algorithms—**Logistic Regression, K-Nearest Neighbors (KNN), and Decision Tree**—were trained and evaluated using performance metrics such as Accuracy, Precision, Recall, F1-Score, Confusion Matrix, and Classification Report. Since Logistic Regression and KNN are sensitive to feature scales, **StandardScaler** was applied before training these models.

After comparing the performance of all models, the best-performing model was selected and optimized using **GridSearchCV** to determine the most suitable hyperparameters. Finally, **feature importance analysis** was performed to identify the resume attributes that had the greatest influence on the hiring prediction.

Project Workflow



Summary

This methodology follows a complete end-to-end Machine Learning pipeline, beginning with data preprocessing and ending with model evaluation and interpretation. Each stage contributes to building a reliable hiring prediction system while ensuring that the selected model is evaluated and optimized using standard machine learning practices.

EXPLORATORY DATA ANALYSIS (EDA)

Exploratory Data Analysis (EDA) was performed to understand the structure and characteristics of the hiring dataset before building the machine learning models. The analysis included examining the dataset information, summary statistics, missing values, duplicate records, and the distribution of both categorical and numerical features. This step helped identify data quality issues and provided insights into the variables influencing the hiring prediction.

The `info()` function was used to examine the data types and confirm the number of non-null values in each column. The `describe()` function provided summary statistics for numerical features such as **Experience (Years)**, **Salary Expectation (\$)**, **Projects Count**, and **AI Score (0–100)**.

The dataset was checked for missing values using `isnull().sum()`. It was observed that only the **Certifications** column contained missing values, while all other columns were complete. These missing values were handled during the data preprocessing stage. Duplicate records were also checked, and no significant duplicates affecting the analysis were found.

The distribution of the target variable, **Recruiter Decision**, was analyzed to understand the balance between hired and rejected candidates. In addition, categorical variables such as **Skills**, **Education**, **Certifications**, and **Job Role** were explored using frequency counts to understand their distribution across the dataset.

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 11 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   Resume_ID             1000 non-null   int64  
 1   Name                  1000 non-null   object  
 2   Skills                1000 non-null   object  
 3   Experience (Years)     1000 non-null   int64  
 4   Education              1000 non-null   object  
 5   Certifications         726 non-null    object  
 6   Job Role              1000 non-null   object  
 7   Recruiter Decision     1000 non-null   object  
 8   Salary Expectation ($) 1000 non-null   int64  
 9   Projects Count         1000 non-null   int64  
10  AI Score (0-100)       1000 non-null   int64  
dtypes: int64(5), object(6)
memory usage: 86.1+ KB
```

```
df.describe()
```

	Resume_ID	Experience (Years)	Salary Expectation (\$)	Projects Count	AI Score (0-100)
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000
mean	500.500000	4.896000	79994.486000	5.13300	83.950000
std	288.819436	3.112695	23048.472549	3.23137	20.983036
min	1.000000	0.000000	40085.000000	0.00000	15.000000
25%	250.750000	2.000000	60415.750000	2.00000	70.000000
50%	500.500000	5.000000	79834.500000	5.00000	100.000000
75%	750.250000	8.000000	99583.250000	8.00000	100.000000
max	1000.000000	10.000000	119901.000000	10.00000	100.000000

```
df.isnull().sum()
```

	0
Resume_ID	0
Name	0
Skills	0
Experience (Years)	0
Education	0
Certifications	274
Job Role	0
Recruiter Decision	0
Salary Expectation (\$)	0
Projects Count	0
AI Score (0-100)	0

dtype: int64

```
df["Recruiter Decision"].value_counts()
```

	count
Recruiter Decision	
Hire	812
Reject	188

dtype: int64

Explore Categorical Columns

```
for col in categorical_columns:
    print("="*50)
    print(col)
    print(df[col].value_counts())
    print()
```

```
... =====
Name
Name
Sarah Jones      3
Crystal Parker   2
James Allen      2
Paul Wallace     2
John Wilson      2
..
Jose Kirk        1
John Alvarez     1
Austin Saunders  1
Shannon Page     1
Abigail Brock    1
Name: count, Length: 989, dtype: int64

=====
Skills
Skills
Python, Machine Learning    11
SQL, Java                   11
Ethical Hacking, Networking  11
SQL, Python                 10
SQL, Machine Learning       10
..
```

```
Ethical Hacking, Networking, Cybersecurity, Linux    1
C++, SQL, Java, React                               1
Networking, Linux, Cybersecurity                     1
Linux, Cybersecurity, Networking, Ethical Hacking    1
NLP, Pytorch, TensorFlow, Python                    1
Name: count, Length: 238, dtype: int64
```

```
=====
Education
Education
B.Sc      205
MBA        202
B.Tech     200
M.Tech     198
PhD        195
Name: count, dtype: int64
```

```
=====
Certifications
Certifications
Deep Learning Specialization    255
AWS Certified                   249
Google ML                      222
Name: count, dtype: int64
```

```
=====
Job Role
Job Role
AI Researcher      257
Data Scientist     255
Cybersecurity Analyst  255
Software Engineer  233
Name: count, dtype: int64
```

```
=====
Recruiter Decision
Recruiter Decision
Hire      812
Reject    188
Name: count, dtype: int64
```

Summary

Exploratory Data Analysis provided a clear understanding of the dataset and ensured that it was suitable for machine learning. It helped identify missing values, verify data quality, and understand the distribution of important features, forming the foundation for effective data preprocessing and model development.

DATA PREPROCESSING

Data preprocessing is an essential step in Machine Learning that prepares raw data for model training by improving its quality and consistency. In this project, several preprocessing techniques were applied to ensure that the dataset was suitable for building an accurate hiring prediction model.

Initially, the dataset was examined for missing values. It was observed that only the **Certifications** column contained missing values. These missing values were replaced with **"No Certification"** to preserve all candidate records while maintaining data consistency.

The **Resume_ID** and **Name** columns were removed from the dataset because they are unique identifiers and do not contribute to predicting recruiter decisions. Including such columns could introduce unnecessary information into the model without improving its predictive performance.

Since Machine Learning algorithms require numerical input, categorical variables including **Skills**, **Education**, **Certifications**, **Job Role**, and **Recruiter Decision** were converted into numerical values using **Label Encoding**. This transformation enabled the algorithms to process the categorical information efficiently.

To prevent **data leakage**, the **AI Score (0–100)** feature was excluded from the training data. This feature was found to have an almost direct relationship with the target variable and could lead to unrealistically high prediction accuracy. Removing it allowed the models to learn meaningful patterns from candidate attributes such as skills, experience, education, certifications, salary expectations, and project count.

Finally, the processed dataset was divided into **training (80%)** and **testing (20%)** sets using a **stratified train-test split**, ensuring that the distribution of hiring decisions remained consistent in both sets.

```
Fill Missing Values

df["Certifications"] = df["Certifications"].fillna("No Certification")

# Verify
df.isnull().sum()

***      0
Resume_ID  0
Name       0
Skills     0
Experience (Years)  0
Education  0
Certifications  0
Job Role   0
Recruiter Decision  0
Salary Expectation ($)  0
Projects Count  0
AI Score (0-100)  0
dtype: int64
```

Encode Categorical Variables

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

categorical_cols = [
    "Skills",
    "Education",
    "Certifications",
    "Job Role",
    "Recruiter Decision"
]

for col in categorical_cols:
    df[col] = le.fit_transform(df[col])

df.head()
```

	Skills	Experience (Years)	Education	Certifications	Job Role	Recruiter Decision	Salary Expectation (\$)	Projects Count	AI Score (0-100)
0	226	10	0	3	0	0	104895	8	100
1	32	10	3	2	2	0	113002	1	100
2	46	1	3	1	1	0	71766	7	70
3	152	7	1	0	0	0	46848	0	95
4	221	4	4	3	3	0	87441	9	100

Feature Selection

Separate Features and Target

```
x = df.drop(["Recruiter Decision", "AI Score (0-100)"], axis=1)
y = df["Recruiter Decision"]

print(x.shape)
print(y.shape)
```

```
... (1000, 7)
(1000,)
```

Train-Test Split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    x,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print(X_train.shape)
print(X_test.shape)
```

```
... (800, 7)
(200, 7)
```

Summary

The preprocessing stage transformed the raw recruitment dataset into a clean and machine-learning-ready format. By handling missing values, removing unnecessary attributes, encoding categorical variables, preventing data leakage, and creating balanced training and testing datasets, the data was successfully prepared for model development and evaluation.

MODEL BUILDING

After completing data preprocessing, multiple Machine Learning classification algorithms were trained and evaluated to predict whether a candidate would be **Hired** or **Rejected**. Each model was trained using the training dataset and evaluated on the testing dataset using performance metrics such as **Accuracy**, **Precision**, **Recall**, **F1-Score**, **Confusion Matrix**, and **Classification Report**. The objective was to compare different algorithms and identify the model that produced the most reliable predictions.

8.1 Logistic Regression (Without Scaling)

Logistic Regression is a supervised machine learning algorithm widely used for binary classification problems. It estimates the probability of an instance belonging to a particular class using the logistic (sigmoid) function. In this project, Logistic Regression was initially trained on the original dataset without applying feature scaling. This served as the baseline model for comparing the effect of scaling and other classification algorithms.

Evaluation Metrics Used:

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix
- Classification Report

```
... Accuracy : 0.955
Precision: 0.8717948717948718
Recall   : 0.8947368421052632
F1 Score : 0.8831168831168831

Confusion Matrix
[[157  5]
 [ 4 34]]

Classification Report
      precision    recall  f1-score   support

     0       0.98       0.97       0.97       162
     1       0.87       0.89       0.88        38

   accuracy       0.95       200
  macro avg       0.92       0.93       0.93       200
 weighted avg       0.96       0.95       0.96       200
```

8.2 Logistic Regression (With Standard Scaling)

Since Logistic Regression performs better when numerical features are on a similar scale, **StandardScaler** was applied to standardize the training and testing datasets. A second

Logistic Regression model was trained using the scaled data, and its performance was compared with the unscaled model.

Scaling helps improve numerical stability and ensures that features with larger values do not dominate the learning process.

```
*** Accuracy : 0.955
Precision: 0.8717948717948718
Recall   : 0.8947368421052632
F1 Score : 0.8831168831168831

Confusion Matrix
[[157  5]
 [ 4 34]]

Classification Report
      precision    recall  f1-score   support

     0       0.98       0.97       0.97        162
     1       0.87       0.89       0.88         38

   accuracy       0.95       0.95       0.95        200
  macro avg       0.92       0.93       0.93        200
 weighted avg       0.96       0.95       0.96        200
```

8.3 K-Nearest Neighbors (KNN)

K-Nearest Neighbors (KNN) is a distance-based classification algorithm that predicts the class of a data point by considering the majority class among its nearest neighbors. Because KNN relies on distance calculations, it was trained using the **scaled dataset**.

The value of **K = 5** was used during model training. The model was evaluated using the same classification metrics to compare its performance with Logistic Regression and Decision Tree.

Evaluate KNN

```
print("Accuracy :", accuracy_score(y_test, y_pred_knn))
print("Precision:", precision_score(y_test, y_pred_knn))
print("Recall   :", recall_score(y_test, y_pred_knn))
print("F1 Score :", f1_score(y_test, y_pred_knn))

print("\nConfusion Matrix")
print(confusion_matrix(y_test, y_pred_knn))

print("\nClassification Report")
print(classification_report(y_test, y_pred_knn))
```

```
*** Accuracy : 0.915
Precision: 0.8181818181818182
Recall   : 0.7105263157894737
F1 Score : 0.7605633802816901

Confusion Matrix
[[156  6]
 [11 27]]

Classification Report
      precision    recall  f1-score   support

     0       0.93       0.96       0.95        162
     1       0.82       0.71       0.76         38

   accuracy       0.92       0.92       0.92        200
  macro avg       0.88       0.84       0.85        200
 weighted avg       0.91       0.92       0.91        200
```

8.4 Decision Tree

Decision Tree is a supervised learning algorithm that classifies data by creating a tree-like structure based on feature values. It recursively splits the dataset into smaller subsets to maximize the separation between classes.

Unlike Logistic Regression and KNN, Decision Tree does not require feature scaling because it makes decisions based on feature thresholds rather than distance calculations.

After comparing all the models, the **Decision Tree** model was selected for further optimization using **GridSearchCV** because it achieved the best performance among the evaluated algorithms.

Evaluate Decision Tree

```
print("Accuracy :", accuracy_score(y_test, y_pred_dt))
print("Precision:", precision_score(y_test, y_pred_dt))
print("Recall   :", recall_score(y_test, y_pred_dt))
print("F1 Score :", f1_score(y_test, y_pred_dt))

print("\nConfusion Matrix")
print(confusion_matrix(y_test, y_pred_dt))

print("\nClassification Report")
print(classification_report(y_test, y_pred_dt))
```

```
... Accuracy : 0.965
Precision: 0.8974358974358975
Recall   : 0.9210526315789473
F1 Score : 0.9090909090909091
```

```
Confusion Matrix
[[158  4]
 [ 3 35]]
```

Classification Report				
	precision	recall	f1-score	support
0	0.98	0.98	0.98	162
1	0.90	0.92	0.91	38
accuracy			0.96	200
macro avg	0.94	0.95	0.94	200
weighted avg	0.97	0.96	0.97	200

Summary

Four classification algorithms were implemented and evaluated in this project. Logistic Regression was trained with and without feature scaling to analyze the impact of normalization, while KNN and Decision Tree provided alternative classification approaches. The performance of all models was compared using standard evaluation metrics, and the best-performing model was selected for further optimization through hyperparameter tuning.

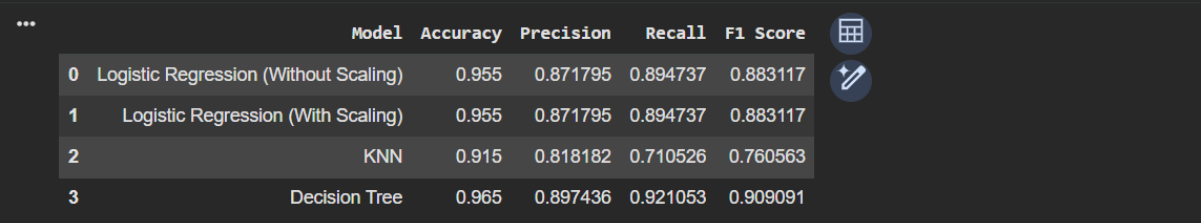
MODEL COMPARISON AND HYPERPARAMETER TUNING

To identify the most suitable machine learning algorithm for hiring prediction, the performance of Logistic Regression (Without Scaling), Logistic Regression (With Scaling), K-Nearest Neighbors (KNN), and Decision Tree was compared using standard classification metrics, including Accuracy, Precision, Recall, and F1-Score. These metrics provide a comprehensive evaluation of each model's prediction capability and help determine the most reliable classifier.

After evaluating all models, the model with the best overall performance was selected for further optimization. To improve its predictive capability and generalization performance, GridSearchCV was applied. GridSearchCV systematically searches different combinations of hyperparameters using cross-validation and selects the combination that produces the best performance based on the chosen evaluation metric.

The optimized model was then retrained using the best hyperparameters and evaluated on the testing dataset. This process ensured that the final model achieved improved performance while reducing the risk of overfitting.

Model Comparison



	Model	Accuracy	Precision	Recall	F1 Score
0	Logistic Regression (Without Scaling)	0.955	0.871795	0.894737	0.883117
1	Logistic Regression (With Scaling)	0.955	0.871795	0.894737	0.883117
2	KNN	0.915	0.818182	0.710526	0.760563
3	Decision Tree	0.965	0.897436	0.921053	0.909091

Best Model Selection

Based on the comparison of all evaluation metrics, the Decision Tree model was selected for further optimization because it demonstrated the best overall classification performance for the hiring prediction task.

Hyperparameter Tuning

To further improve the Decision Tree model, GridSearchCV was applied with multiple combinations of hyperparameters, including:

- Criterion (Gini / Entropy)
- Maximum Depth
- Minimum Samples Split

- Minimum Samples Leaf

The best parameter combination identified by GridSearchCV was used to train the final optimized model.

```
Best Parameters

+ Code + Text

print("Best Parameters:")
print(grid.best_params_)

print("\nBest F1 Score:")
print(grid.best_score_)

... Best Parameters:
{'criterion': 'entropy', 'max_depth': 10, 'min_samples_leaf': 1, 'min_samples_split': 2}

Best F1 Score:
0.8934449952797154

Train Best Model

best_dt = grid.best_estimator_

y_pred_best = best_dt.predict(X_test)
```

Optimized Model Performance

After hyperparameter tuning, the optimized Decision Tree model was evaluated again using the testing dataset. The final evaluation included:

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix
- Classification Report

```
... Accuracy : 0.945
Precision: 0.8648648648648649
Recall : 0.8421052631578947
F1 Score : 0.8533333333333334

Confusion Matrix
[[157  5]
 [ 6 32]]

Classification Report
      precision    recall  f1-score   support

0       0.96       0.97       0.97       162
1       0.86       0.84       0.85        38

accuracy      0.94       0.94       0.94       200
macro avg     0.91       0.91       0.91       200
weighted avg   0.94       0.94       0.94       200
```

Summary

The comparison of multiple machine learning models helped identify the most effective algorithm for the hiring prediction task. Hyperparameter tuning further improved the selected model by determining the optimal configuration, resulting in a more reliable and accurate prediction system.

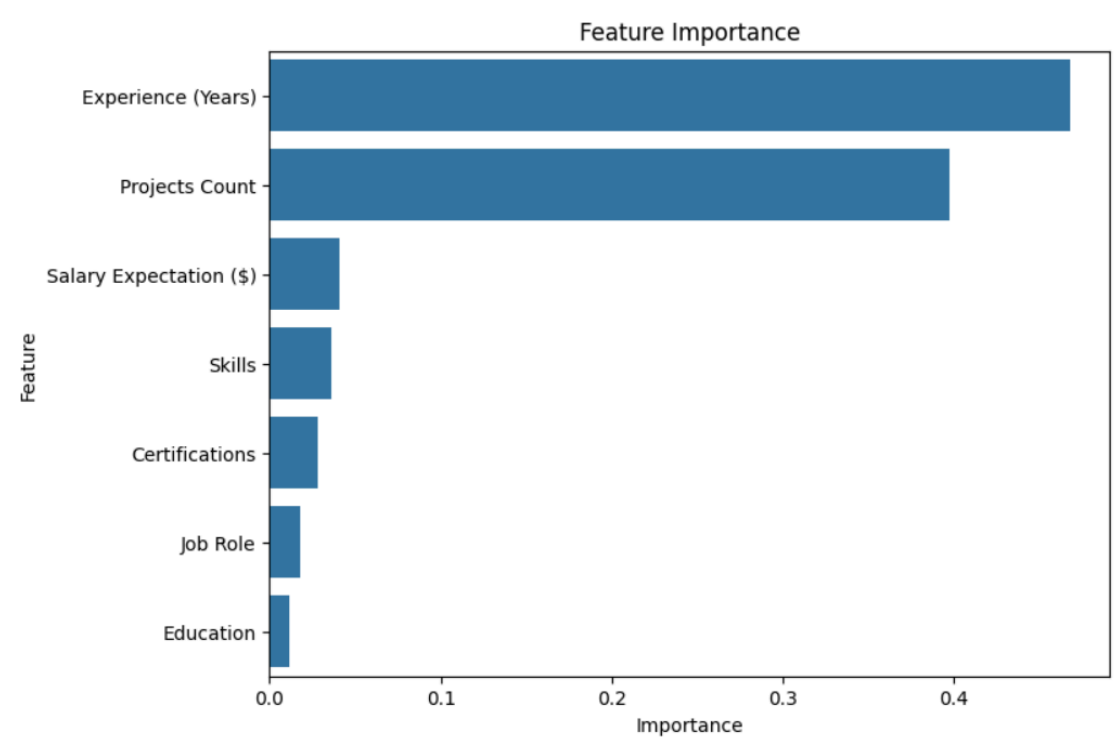
FEATURE IMPORTANCE ANALYSIS

Feature Importance Analysis was performed to identify the resume attributes that had the greatest influence on the hiring prediction. This analysis helps in understanding how the Decision Tree model makes its predictions and which candidate characteristics contribute most to the recruitment decision.

To obtain reliable and meaningful results, the **AI Score (0–100)** feature was excluded from the training data before model development. This prevented **data leakage**, ensuring that the model learned from genuine resume information rather than relying on a single feature that was highly correlated with the target variable.

The Decision Tree model assigned an importance score to each feature based on its contribution to reducing classification error during training. Features with higher importance values had a greater impact on the hiring prediction, while features with lower importance contributed less to the model's decisions.

The feature importance graph provides a visual representation of the relative contribution of each feature. This analysis helps recruiters and data analysts understand which resume characteristics influence hiring decisions the most and improves the interpretability of the machine learning model.



Observation

The feature importance analysis showed that the Decision Tree model primarily relied on meaningful resume-related attributes such as **experience, education, skills, certifications, projects, salary expectation, and job role** to make predictions. Removing the **AI Score (0–100)** feature ensured that the model learned from candidate information rather than depending on a feature that could introduce data leakage. This resulted in a more realistic and reliable hiring prediction model.

Summary

Feature Importance Analysis improved the interpretability of the machine learning model by identifying the most influential factors affecting hiring decisions. It also demonstrated the importance of selecting appropriate features and avoiding data leakage when developing AI-based recruitment systems.

CONCLUSION

This project successfully developed an AI-Based Hiring Prediction System using Machine Learning techniques to predict whether a candidate is likely to be Hired or Rejected based on resume-related information. The project followed a complete end-to-end machine learning workflow, including data exploration, preprocessing, model development, evaluation, hyperparameter tuning, and feature importance analysis.

Three machine learning algorithms—Logistic Regression, K-Nearest Neighbors (KNN), and Decision Tree—were trained and evaluated using performance metrics such as Accuracy, Precision, Recall, F1-Score, Confusion Matrix, and Classification Report. The best-performing model was further optimized using GridSearchCV, resulting in improved predictive performance.

During preprocessing, unnecessary columns such as Resume_ID and Name were removed, categorical variables were encoded, and the AI Score (0–100) feature was excluded to prevent data leakage. This ensured that the model learned from meaningful candidate information rather than relying on a feature directly related to the target variable, resulting in a more reliable and realistic prediction system.

Overall, the project demonstrates how Machine Learning can assist recruiters by automating resume screening, reducing manual effort, and supporting data-driven hiring decisions. It also highlights the importance of proper data preprocessing, model evaluation, and feature selection in developing trustworthy AI-based recruitment systems.

FUTURE SCOPE

The AI-Based Hiring Prediction System can be further improved by incorporating advanced machine learning and natural language processing techniques. Some possible future enhancements include:

- Use TF-IDF, Word2Vec, or BERT embeddings to analyze resume text instead of treating skills as simple categorical values.
- Apply MultiLabelBinarizer or One-Hot Encoding for the Skills column to represent individual skills more accurately.
- Train and evaluate the model using real-world recruitment datasets collected from recruitment platforms.
- Develop a web application using Flask, Django, or Streamlit to allow recruiters to upload resumes and receive hiring predictions.
- Integrate the system with Applicant Tracking Systems (ATS) for automated resume screening.
- Explore advanced machine learning models such as Random Forest, XGBoost, or LightGBM to improve prediction performance.

REFERENCES

1. Scikit-learn Developers. *Scikit-learn Documentation*. <https://scikit-learn.org/>
2. Pandas Development Team. *Pandas Documentation*. <https://pandas.pydata.org/>
3. NumPy Developers. *NumPy Documentation*. <https://numpy.org/>
4. Matplotlib Development Team. *Matplotlib Documentation*. <https://matplotlib.org/>
5. Google. *Google Colaboratory Documentation*. <https://colab.research.google.com/>
6. 3Skill Internship Program – AI-Based Hiring Prediction Dataset (Internship Project Dataset).